

GCF Sales CRM — Claude Code Build Brief

Oct 8, 2026 · @Kenji Sakuramoto

How to use this brief

Create four free accounts, connect four MCP servers, save this doc into the repo, then paste the prompt at the bottom into Claude Code.

1. Create free accounts: Cloudflare, Supabase (two projects, `gcf-crm-dev` and `gcf-crm-prod`, both in Singapore), Resend, GitHub (private repo `gcf-crm`).
2. Open Claude Code in the empty repo and connect the MCP servers below, then run `/mcp` to sign in to each.
3. Export this doc as Markdown and save it as `docs/BRIEF.md` in the repo.
4. Paste the prompt from the last section. Claude Code plans first, you approve, then it builds phase by phase.

MCP servers to connect

Server	Priority	What it does in this build	Command
Supabase	Core	Runs migrations, inspects schema, checks RLS advisors on the dev project	<code>claude mcp add --scope project --transport http supabase "https://mcp.supabase.com/mcp?project_ref=<DEV_REF>"</code>
Context7	Core	Pulls current, version-specific docs so Claude Code doesn't guess APIs	<code>claude mcp add --transport http context7 https://mcp.context7.com/mcp</code>
shadcn	Core	Searches and installs shadcn/ui components and blocks	<code>npx shadcn@latest mcp init --client claude</code>

Server	Priority	What it does in this build	Command
Playwright	Core	Drives a real browser to verify screens and run E2E checks	<code>claude mcp add playwright -- npx @playwright/mcp@latest</code>
Cloudflare docs + observability	Optional	Workers/Access docs, deploy logs	<code>claude mcp add --transport http cloudflare-docs https://docs.mcp.cloudflare.com/mcp</code>
Resend	Optional	Domain verification and test sends	<code>claude mcp add --transport http resend https://mcp.resend.com/mcp</code>
GitHub	Optional	PRs, Actions (backup workflow)	<code>claude mcp add --transport http github https://api.githubcopilot.com/mcp/</code>

Two guardrails. Scope the Supabase MCP to the `dev` project only and never point it at production; Supabase states its MCP server is for development and testing. On native Windows, wrap the `npx`-based servers as `cmd /c npx ...` so they start.

The CEO brief

Build Golden Cap Farm a simple sales CRM for 3–4 people: win new chef accounts, keep existing accounts ordering, and run every visit, tasting and follow-up from one calendar. No invoicing.

Users: two roles. Admin (1 person) can do everything, including users, settings and full exports. Sales (2–3 people) can do everything else.

Success criteria

- A new lead is logged in under 30 seconds.
- Every account and open deal has a next-touch date; nothing goes silent.
- The Monday pipeline review takes 10 minutes from one screen.
- Accounts that are ordering less, or have stopped, surface before anyone has to look for them.

- Contacts import from the legacy CRM, Xero or any spreadsheet in under 2 minutes; any list exports in one click.
- \$0/month.

Out of scope for v1: invoices, payments, credit control, statements, order entry, delivery routing, stock. Xero handles money. The CRM only reads a weekly order export from Xero to spot accounts that are slipping.

Non-negotiables

1. Desktop-first for 1280–1920 px screens. Mobile only has to not break.
2. Staff-only access to customer data.
3. Golden Cap house style: green/gold palette, Arial, EN/中文 labels.
4. Free tiers only.

Global best practice for produce sales

Leading produce and foodservice distributors win on eight habits; the CRM builds each one in so it happens by default, not by discipline.

Habit	Why it works	How the CRM does it
Send a weekly availability list	Chefs plan menus around what is in season; growers' guidance is to give buyers a weekly availability list (NCAT)	Tick "available this week" on products; one click turns it into WhatsApp text, a PDF or an email
Sample, then trial	Distributor reps grow accounts by sampling products with chefs (Performance Foodservice)	Sample/tasting is a pipeline stage; chef feedback is required to move on
Cover accounts by potential	Bain found rep time tracks share of wallet, and says coverage should follow future potential, not last year's revenue (Bain)	Tier A/B/C from estimated weekly produce spend; each tier has a call cycle (default A weekly, B fortnightly, C monthly) that sets the next-touch date

Habit	Why it works	How the CRM does it
Grow share of wallet by cross-selling	The same Bain study found cross-selling categories was a big driver of share growth (Bain)	Each account lists products they buy and products to pitch, plus our estimated share
Catch quiet churn early	Wholesale accounts leave gradually: fewer reorders, smaller orders, dropped categories (RepSpark , Choco)	Account-health loop compares days since last order with each account's own rhythm and flags dropped regular products
Know the whole kitchen	Chefs change jobs often, so relationships must span the staff (NCAT)	Several contacts per account by role; a "chef moved" action keeps their history and opens a lead at the new venue
Respect the kitchen's clock	Chefs want deliveries outside service, often before 10 a.m. (NCAT)	Delivery window and order cutoff show on every account card and linked calendar event
Fix service failures fast	Short ships and late deliveries cause much of wholesale churn (RepSpark)	Logging a quality or delivery issue creates a 24-hour follow-up task, and repeat issues count against account health

Produce-distribution CRM model

A produce distributor sells to accounts that reorder on a rhythm, so the CRM runs two loops: a deal pipeline that wins the account, then an account-health loop that keeps it ordering. Wholesale CRMs use the same shape: enquiry, samples, quote, first order, ongoing account ([Pipeline CRM](#)).

Deal pipeline (one deal per prospect venue)

#	Stage	Exit rule (enforced before moving on)	Default next action (editable in Settings)
1	Lead	Venue, segment and one contact recorded	First contact within 3 days

#	Stage	Exit rule (enforced before moving on)	Default next action (editable in Settings)
2	Contacted	Products of interest and decision-maker recorded	Book a tasting or sample drop
3	Sample / tasting	Sample date and chef feedback logged	Send quote within 2 days
4	Quote sent	Quote attached; every line at or above the 15% GM floor, or override reason given	Follow up in 3 days
5	Trial order	First order date, delivery days and credit terms set	Check in after the 2nd delivery
6	Won	Account flips to Active; deal closes	Account-health loop takes over
-	Lost	Reason required: price, quality, supply, competitor, no response, other	Optional re-engage date

Account-health loop (after Won)

- **Active:** ordered within its usual interval, computed from the weekly order import.
- **At risk:** no order for 1.5× its usual interval; flagged on the dashboard and in the owner's digest.
- **Dormant:** no order for 30 days; the system opens a re-engagement deal at Contacted.
- **Closed:** account stopped trading, with a reason.

Early-warning flags, shown on the account and in the owner's digest: a regular product missing from the last 3 orders, order value down 30% or more against its 8-week average, or 2 or more quality or delivery issues in 30 days (defaults, editable in Settings).

Fields this industry needs (beyond a generic CRM)

- Segment: fine-dining, Chinese, hotpot, Japanese, hotel, caterer, retail, wholesale (PPWC), online (Shopify).
- Coverage: tier A/B/C, call cycle, next-touch date, estimated weekly produce spend, our estimated share %, products they buy, products to pitch.
- Delivery: delivery days (multi-select), time window, one or more delivery addresses, receiving notes (loading bay, chiller, who signs).

- Ordering: order cutoff time, regular products and usual quantities, usual order interval.
- Terms agreed: billing entity vs trading name, UEN, credit terms (COD, 7, 14, 30 days), Xero contact name to match the weekly order import.
- Pricing: price list per account or tier; every line shows cost, price, markup and GM side by side, because markup ($\text{cost} \times 1.30$) and GM ($\text{price} - \text{cost}$) $\div$ price are not the same number.
- Products: code (CF, QQ, etc.), EN and 中文 names, unit (kg, pack, carton), origin, shelf life in days, seasonality months.
- Contacts by role: owner, head chef, purchaser, accounts payable, receiving.
- Activity types: call, WhatsApp, visit, tasting, sample drop, quote, delivery issue, quality complaint, chef moved, note.

Screens

v1 is seven desktop screens, a command palette and a daily email digest.

Screen	What's on it	Key actions
Today	My touches due and overdue; today's visits, tastings and deliveries; accounts that just turned at-risk; deals stuck 14+ days in a stage	Click any item to open it
Pipeline	Kanban by stage; each card shows venue, est. weekly SGD, owner, next-touch date (red when late), days in stage	Drag to move, which runs the exit-rule check; filter by owner or tier
Accounts	Table with tier, health, last order, next touch, owner; saved views	Bulk change owner or tier; export
Account detail	Left: profile, delivery and terms. Centre: timeline. Right: contacts, products they buy and to pitch, open deal	Log activity, schedule event, mark "chef moved", all without leaving the page

Screen	What's on it	Key actions
Calendar	Month, week, day and agenda views, coloured by type: visit, tasting, sample drop, call, follow-up, delivery	Click a slot to create an event linked to an account; drag to reschedule; recurring events; filter by person
Products	Codes, EN/中文 names, unit, season, "available this week", price tiers with markup and GM	Generate the weekly availability list as WhatsApp text, PDF or email
Import/export and Settings	Upload wizard, saved mappings, import history; users, stages, tiers and call cycles	Undo an import; change rules without code

Everywhere: `Ctrl/Cmd+K` to search or quick-add, EN/中文 toggle, green/gold theme, layout built for 1280–1920 px.

Email (Resend): a 07:30 Singapore-time digest per person (today's events, touches due, accounts turned at-risk, stuck deals), plus an instant email when you're assigned a deal or task or invited to an event. That is well under the free 100 emails/day.

Import

1. Upload a CSV or .xlsx file (handles the legacy CRM's BOM + CRLF format).
2. Pick what it is: accounts and contacts, products, or the weekly Xero order export.
3. Map columns once and save the mapping ("Legacy CRM", "Xero contacts", "Xero orders").
4. Preview with row errors; duplicates match on email, +65 phone or UEN, then skip or update.
5. Commit, and undo the whole batch if it was wrong.

Export

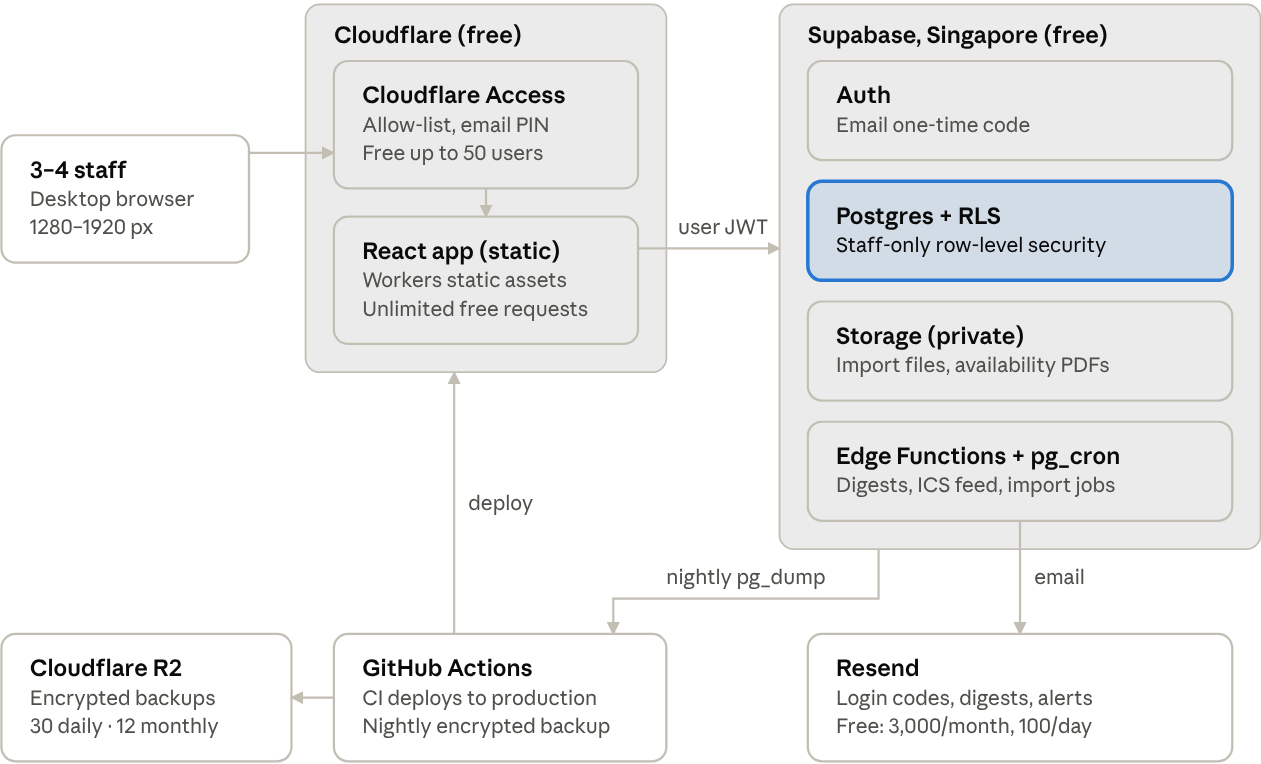
- Any table: CSV (UTF-8 with BOM, so 中文 opens correctly in Excel) or .xlsx, using the current filters and columns.
- Contacts: vCard (.vcf) to load straight into a phone.
- Calendar: .ics file, plus a private subscription link for Google or Outlook Calendar.
- Admin: full export of every table as a ZIP.

Stack decision (2026)

Run a static React app on Cloudflare Workers, with Supabase in Singapore as the database, login and job runner, and Resend for email. It costs \$0/month at 4 users and, unlike Vercel's free plan, is allowed for commercial use.

Staff pass two locks before any data; secrets stay server-side

\$0/month for 3–4 users · all customer data in Supabase Singapore



GCF CRM architecture · 3 zones, 11 parts

The browser holds only the public key and the user's login token; the Resend key and backup credentials live in Supabase and GitHub secrets.

Layer	Pick	Free-tier limit that matters
Hosting	Cloudflare Workers serving a static React app, with Cloudflare Access as the front gate	Static requests free and unlimited (Cloudflare); Access free up to 50 users (CostBench). Vercel's free plan bans commercial use (Vercel)

Layer	Pick	Free-tier limit that matters
App	React 19 + TypeScript + Vite; TanStack Router, Query and Table; shadcn/ui + Tailwind CSS v4; react-hook-form + Zod; dnd-kit for the kanban	All open source (MIT)
Database and login	Supabase in Singapore: Postgres, Auth with email one-time codes, Edge Functions, <code>pg_cron</code>	500 MB database, pauses after 1 week idle, no automatic backups (Supabase)
Email	Resend + React Email, also used as Supabase's login-email sender	3,000/month, 100/day, 1 domain (Resend)
Calendar	FullCalendar v6 with MIT plugins only (day, week, list, drag-and-drop, recurring)	Premium plugins need a paid licence, so they are banned
Import and export	Papa Parse, SheetJS CE, <code>ics</code> , vCard, <code>@react-pdf/renderer</code> for the availability PDF	None
Backup	GitHub Actions nightly <code>pg_dump</code> , encrypted, stored in Cloudflare R2	Needed because Supabase Free keeps no backups

If it outgrows free, Supabase Pro at \$25/month removes the pause and adds daily backups ([Supabase](#)).

Data, access and backups

Twelve tables cover v1; only signed-in staff can touch them, and an encrypted copy leaves Supabase every night.

On every table: `id uuid`, `created_at`, `updated_at`, `created_by`, soft delete via `deleted_at`. Money in SGD as `numeric(12,2)`, times as `timestamptz` shown in Asia/Singapore, phones in E.164 (+65...).

Table	Key columns
<code>profiles</code>	<code>user_id</code> , <code>full_name</code> , role (admin, sales), locale (en, zh), <code>is_active</code>

Table	Key columns
accounts	trading_name, legal_name, uen, segment, tier, health, owner_id, delivery_days, delivery_window, order_cutoff, credit_terms_days, xero_contact_name, est_weekly_spend_sgd, est_share_pct, usual_interval_days, last_order_at, next_touch_at
contacts	account_id, full_name, role, phone_e164, email, is_primary, marketing_consent, consent_at, previous_account_id (set by "chef moved")
deals	account_id, stage, est_weekly_sgd, owner_id, next_action, next_action_at, stage_entered_at, lost_reason
activities	type, account_id, contact_id, deal_id, occurred_at, summary, outcome
events	title, type, starts_at, ends_at, all_day, rrule, account_id, deal_id, owner_id, attendee_ids
tasks	title, due_at, assignee_id, account_id, deal_id, source (manual, rule)
products	code, name_en, name_zh, unit, origin, season_months, available_this_week, cost_sgd
product_prices	product_id, tier (A, B, C, list), price_sgd; generated markup_pct and gm_pct
account_products	account_id, product_id, status (buys, pitch), usual_qty
orders	account_id, order_date, total_sgd, lines (code, qty); filled only by the weekly Xero import, read-only in the app
import_batches	target, file_name, mapping, row counts, committed_at, undone_at

Access rules

1. Cloudflare Access allow-lists the staff emails in front of the app.

2. Login is an emailed one-time code, no passwords. Anyone can switch on an authenticator-app second factor in Settings.
3. Row-level security on every table: active staff read and write everything; only admin manages users and settings, hard-deletes, or runs the full export.
4. The Supabase service key lives only in Edge Function and GitHub secrets, never in the browser.
5. Singapore PDPA: store marketing consent and its date per contact, name a Data Protection Officer in Settings, and keep phone numbers out of emails.

Backups: GitHub Actions runs `pg_dump` nightly, encrypts it with `age` and stores it in Cloudflare R2, keeping 30 daily and 12 monthly copies. Restore into the dev project once a quarter to prove it works.

Build plan

Claude Code builds in four phases, each ending in a gate it must pass before moving on; production changes only through CI.

Team: the main Claude Code session plans and runs three subagents: a builder (schema and screens), a tester (Vitest, Playwright, row-level security tests) and a fresh-context reviewer who checks security and UX at every gate without seeing the builder's reasoning. If your `plan-rls-audit` and `plan-uiux-audit` skills are installed in Claude Code, the reviewer runs them.

1. **Foundation.** Repo, Vite + React + TS, Tailwind, shadcn/ui, Supabase dev project, all 12 tables with row-level security and seed data, email-code login, deploy behind Cloudflare Access, Resend domain. Gate: a staff email signs in on the staging URL; a non-staff email is stopped at Access; security tests pass.
2. **Accounts and pipeline.** Accounts, contacts, account timeline, activities, tasks, kanban with exit rules, `Ctrl/Cmd+K`. Gate: an automated test logs a new lead in under 30 seconds and moves it to Sample/tasting.
3. **Calendar, import and export.** Calendar views, linked and recurring events, .ics export and subscription link; import wizard with mappings, duplicate handling and undo; CSV, .xlsx and vCard export; dry run of the legacy CRM file in dev. Gate: imported counts match the source file, and export-then-reimport changes zero rows.
4. **Health, availability and go-live.** Weekly Xero order import, account-health loop and early-warning flags, tiers and call cycles setting next-touch dates, weekly availability list, digest emails, nightly backup with a restore drill. Gate: a seeded slipping account

shows up in the next digest, and the restore drill passes.

Done means, every phase

- ☐ All tests green in CI
- ☐ Screenshots at 1440×900 and 1280×800 reviewed, no horizontal scroll
- ☐ Production changed only by the CI deploy job
- ☐ Decision log in `docs/BRIEF.md` updated

The paste-ready Claude Code prompt

Paste this as the first message in Claude Code, after the MCP servers are connected and this doc is saved as `docs/BRIEF.md`.

You are the product and engineering team building GCF Sales CRM for Golden Cap Farm, a Singapore specialty mushroom and produce distributor (SFA-licensed importer; produce air-freighted overnight from farms in Yunnan). I am the CEO. Build to the standard of the world's best sales CRMs, but keep it simple: 3-4 internal users, \$0/month.

Read `docs/BRIEF.md` first. It is the source of truth for scope, best practices, fields, data model, stack, phases and gates. Where `BRIEF.md` is silent, choose the simplest option and record it in its decision log.

What this is

A sales CRM only: win new chef accounts, keep existing accounts ordering, and run every visit, tasting and follow-up from one calendar. No invoices, payments, credit control or order entry; Xero handles money. The CRM only imports a weekly Xero order export (read-only) to compute account health.

Fixed stack (do not substitute without asking)

- Static Vite + React 19 + TypeScript app on Cloudflare Workers, behind Cloudflare Access. No Next.js, no Vercel.
- TanStack Router, Query and Table; shadcn/ui + Tailwind CSS v4; react-hook-form + Zod; dnd-kit.
- Supabase in ap-southeast-1 (Singapore): Postgres with row-level security, Auth with emailed one-time codes (no passwords), Edge Functions, `pg_cron`.
- Resend + React Email for all email, including Supabase login emails.
- FullCalendar v6 with MIT plugins only. Never install premium plugins.
- Papa Parse + SheetJS CE for import; CSV (UTF-8 with BOM), .xlsx, .ics, vCard and @react-pdf/renderer for export.
- pnpm, Vitest, Playwright, pgTAP; GitHub Actions for CI, deploys and nightly

encrypted backups to Cloudflare R2.

Must-haves

1. Desktop-first for 1280-1920 px: dense tables, side panels, keyboard shortcuts, Ctrl/Cmd+K. Mobile only has to not break.
2. Pipeline: Lead -> Contacted -> Sample/tasting -> Quote sent -> Trial order -> Won, plus Lost with a required reason. Enforce the exit rules in BRIEF.md.
3. Account health after Won: Active / At risk (no order for 1.5x its usual interval) / Dormant (30 days; auto-open a re-engagement deal) / Closed, plus the early-warning flags in BRIEF.md.
4. Tiers A/B/C with call cycles that set each account's next-touch date. Every account and open deal always has a next touch.
5. Calendar: month/week/day/agenda, events linked to accounts, recurring events, drag to reschedule, .ics export and a private subscription link.
6. Weekly availability list from products marked "available this week", as WhatsApp text, PDF or email.
7. Import with column mapping, saved mappings, duplicate handling and batch undo (legacy CRM files are UTF-8 with BOM and CRLF). Every list exports to CSV and .xlsx; contacts also to vCard.
8. A 07:30 Singapore-time digest email per person.
9. Price lines show cost, price, markup and GM. $GM = (price - cost) / price$. $Markup = (price - cost) / cost$. Warn below a 15% GM.
10. Golden Cap house style: green/gold palette, Arial, EN/中文 from day one. Product codes carry EN and 中文 names, e.g. CF = cordyceps flower 虫草花, QQ = black skin 黑皮鸡枞.
11. Row-level security on every table, staff-only; the service key never reaches the browser.
12. Free tiers only. If something needs a paid plan, stop and tell me.

How to work

- Start in plan mode: read BRIEF.md, then propose the repo structure, migrations, route map and the Phase 1 task list. Wait for my approval.
- Use subagents as described in BRIEF.md: builder, tester, and a fresh-context reviewer at every gate.
- Pull current library docs with Context7 before coding against a library; add UI with the shadcn MCP; check every screen with the Playwright MCP at 1440x900 and 1280x800.
- The Supabase MCP is connected to the dev project only. Never touch production; only the CI deploy job changes it.
- One phase at a time. At each gate: run all tests, show screenshots, list changes and open questions, then wait.
- Ask me only when a decision is irreversible or costs money. Otherwise decide, log it in BRIEF.md and continue.

```
## First deliverable
```

```
The Phase 1 (Foundation) plan, plus the exact accounts, keys and DNS records  
I need to create, in order. No code until I approve.
```

Sources

- [NCAT: Tips for selling to restaurants](#)
- [Bain: Is that customer worth your time?](#)
- [RepSpark: Using data to prevent B2B buyer churn](#)
- [RepSpark: Spotting valuable accounts at risk of switching](#)
- [Choco: 5 customer data signals food distributors should track](#)
- [Performance Foodservice sales representative role](#)
- [Pipeline CRM for wholesalers and distributors](#)
- [Vercel Fair Use Guidelines](#)
- [Cloudflare Workers static assets billing](#)
- [Cloudflare Zero Trust pricing \(CostBench\)](#)
- [Supabase pricing](#)
- [Supabase regions](#)
- [Supabase MCP docs](#)
- [Resend account quotas and limits](#)
- [shadcn MCP server](#)
- [Cloudflare MCP servers](#)
- [Context7 and Playwright MCP setup \(Bito\)](#)